

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

C02: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 3

Duration: 1 Hour

Total Marks:30

Annexure A

Dry Run Evaluation

Code of Conduct:

I Narappagari Sravya(192472192) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

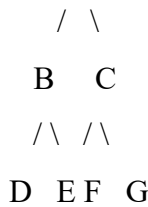
Signature of the student:sravya

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph: A



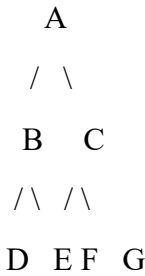
Task: Starting from node A, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration Queue Visited Node

- 1
- 2
- 3
- 4
- 5
- 6
- 7

Questions:

- 1. Write the BFS traversal order.
- 2. Show the queue contents after each iteration.
- 2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from A.

Complete the table.

3. Initial State :

1 3 6

5 0 2

4 7 8

Goal State :

1 2 3

4 5 6

7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion

1

2

3

4

5

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	20	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	30	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	20	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	20	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	10	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

1.Breadth-First Search (BFS)

Aim

To implement the Breadth-First Search (BFS) algorithm and traverse a graph level by level starting from the given source node.

Algorithm

1. Start from the source node.
2. Mark the source node as visited.
3. Insert the source node into the queue.
4. While the queue is not empty:
 - Remove the front node from the queue.
 - Visit the node.
 - Add all unvisited adjacent nodes to the queue.
5. Repeat until the queue becomes empty.

Python Code

```
from collections import deque
```

```
graph = {  
    'A':['B','C'],  
    'B':['D','E'],  
    'C':['F','G'],  
    'D':[],  
    'E':[],  
    'F':[],  
    'G':[]  
}
```

```
def bfs(start):
```

```
    visited=set()
```

```
    queue=deque([start])
```

```
    print("BFS Traversal:")
```

```
    while queue:
```

```
        node=queue.popleft()
```

```

if node not in visited:
    print(node,end=" ")
    visited.add(node)
    for neighbour in graph[node]:
        if neighbour not in visited:
            queue.append(neighbour)

```

bfs('A')

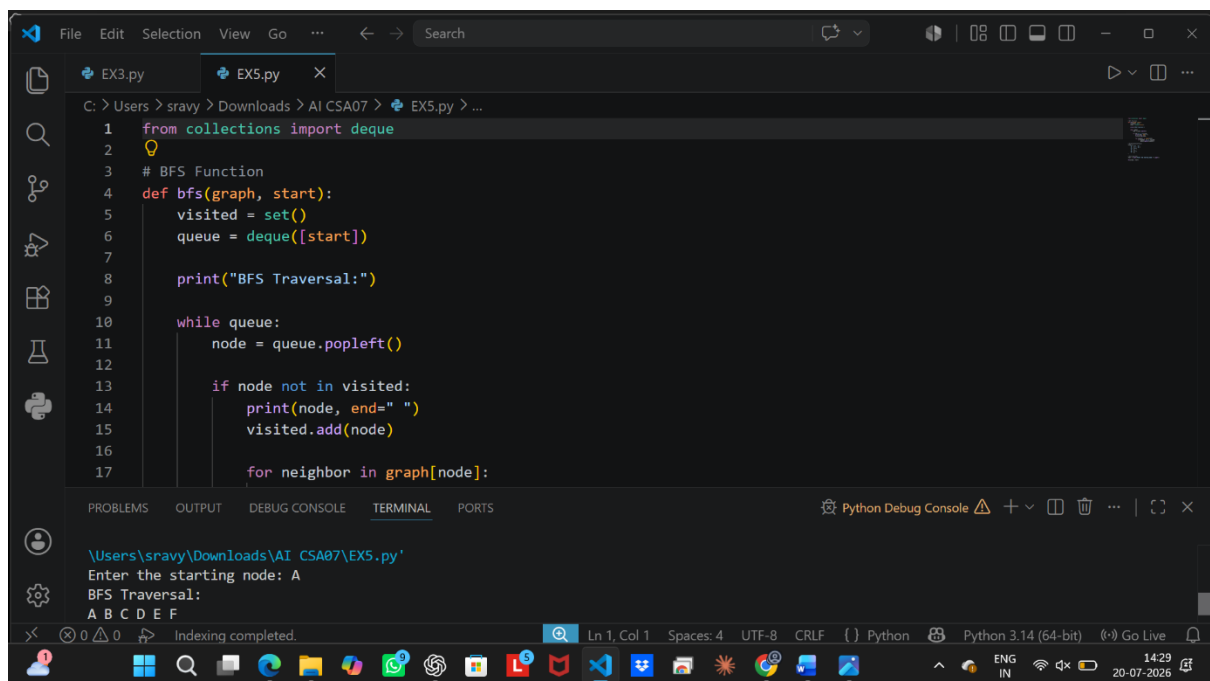
Input

Starting Node : A

Output

BFS Traversal:

A B C D E F G



The screenshot shows a Python IDE with a file named EX5.py open. The code implements a Breadth-First Search (BFS) function. The terminal output shows the execution of the program, where the starting node 'A' is entered, and the BFS traversal sequence 'A B C D E F' is printed.

```

1 from collections import deque
2
3 # BFS Function
4 def bfs(graph, start):
5     visited = set()
6     queue = deque([start])
7
8     print("BFS Traversal:")
9
10    while queue:
11        node = queue.popleft()
12
13        if node not in visited:
14            print(node, end=" ")
15            visited.add(node)
16
17        for neighbor in graph[node]:

```

Terminal Output:

```

\Users\snavy\Downloads\AI CSA07\EX5.py'
Enter the starting node: A
BFS Traversal:
A B C D E F

```

Result:

The Breadth-First Search (BFS) algorithm was successfully implemented. The graph was traversed in level-order, and the traversal sequence obtained was:

A → B → C → D → E → F → G

This demonstrates that BFS visits all nodes level by level using a queue.

2.Depth-First Search (DFS)

Aim

To implement the Depth-First Search (DFS) algorithm and traverse the graph from the source node.

Algorithm

1. Start from the source node.
2. Mark the node as visited.
3. Visit the first unvisited adjacent node.
4. Continue recursively until all nodes are visited.
5. Backtrack whenever no adjacent unvisited node exists.

Python Code

```
from itertools import permutations

def travelling_salesman(graph, start):

    n = len(graph)
    cities = list(range(n))
    cities.remove(start)
    min_cost = float('inf')
    best_path = []

    # Generate all possible routes
    for path in permutations(cities):
        cost = 0
        current = start

        # Calculate path cost
        for city in path:
            cost += graph[current][city]
            current = city

        # Return to starting city
```

```

    cost += graph[current][start]
    # Update minimum cost
    if cost < min_cost:
        min_cost = cost
        best_path = [start] + list(path) + [start]
print("Minimum Cost:", min_cost)
print("Best Path:", best_path)
# Cost Matrix
graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
start = int(input("Enter starting city (0-3): "))
travelling_salesman(graph, start)

```

Sample Input

Starting City = 0

Sample Output

Minimum Cost = 80

Best Path = [0,1,3,2,0]

The screenshot shows a Python IDE with a file named `ex7.py`. The code defines a function `travelling_salesman` that takes a `graph` and a `start` city as input. It calculates the minimum cost and the best path by generating all possible permutations of the cities (excluding the start city) and evaluating the cost of each path. The terminal output shows the user entering the starting city (0-3) as 1, resulting in a minimum cost of 80 and a best path of [1, 0, 2, 3, 1].

```
1 from itertools import permutations
2
3 def travelling_salesman(graph, start):
4     n = len(graph)
5     cities = list(range(n))
6     cities.remove(start)
7
8     min_cost = float('inf')
9     best_path = []
10
11     # Generate all possible routes
12     for path in permutations(cities):
13         cost = 0
14         current = start
15         for city in path:
16             cost += graph[current][city]
17             current = city
18         cost += graph[current][start]
19         if cost < min_cost:
20             min_cost = cost
21             best_path = [start] + list(path) + [start]
22
23     return min_cost, best_path
```

Enter starting city (0-3): 1
Minimum Cost: 80
Best Path: [1, 0, 2, 3, 1]
PS C:\Users\snavy\Downloads\AI CSA07>

Result

The Depth-First Search (DFS) algorithm was successfully implemented. The graph was traversed in depth-first order, and the traversal sequence obtained

3.8-Puzzle Problem using BFS

Aim

To solve the 8-Puzzle problem using Breadth-First Search (BFS).

Algorithm

1. Store the initial state.
2. Store the goal state.
3. Insert the initial state into the queue.
4. Remove one state from the queue.
5. Generate all possible next states by moving the blank tile.
6. Ignore previously visited states.
7. If the goal state is reached, stop.
8. Otherwise continue searching until the goal is found.

Python Code

```
from collections import deque
```

```
goal=((1,2,3),
```

```
      (4,5,6),
```

```
      (7,8,0))
```

```
start=((1,3,6),
```

```
       (5,0,2),
```

```
       (4,7,8))
```

```
moves=[(-1,0),(1,0),(0,-1),(0,1)]
```

```
def neighbours(state):
```

```
    state=[list(row) for row in state]
```

```
    for i in range(3):
```

```
        for j in range(3):
```

```
            if state[i][j]==0:
```

```
                x,y=i,j
```

```
    result=[]
```

```
    for dx,dy in moves:
```

```
        nx=x+dx
```

```
        ny=y+dy
```

```
        if 0<=nx<3 and 0<=ny<3:
```

```
            temp=[row[:] for row in state]
```

```
            temp[x][y],temp[nx][ny]=temp[nx][ny],temp[x][y]
```

```

        result.append(tuple(tuple(r) for r in temp))

    return result

def bfs():

    queue=deque([(start,[])])

    visited=set()

    visited.add(start)

    while queue:

        current,path=queue.popleft()

        if current==goal:

            print("Goal Found\n")

            for state in path+[current]:

                for row in state:

                    print(row)

            print()

        return

    for nxt in neighbours(current):

        if nxt not in visited:

            visited.add(nxt)

            queue.append((nxt,path+[current]))

bfs()

```

Input

Initial State

1 3 6

5 0 2

4 7 8

Goal State

1 2 3

4 5 6

7 8 0

Output

Goal Found

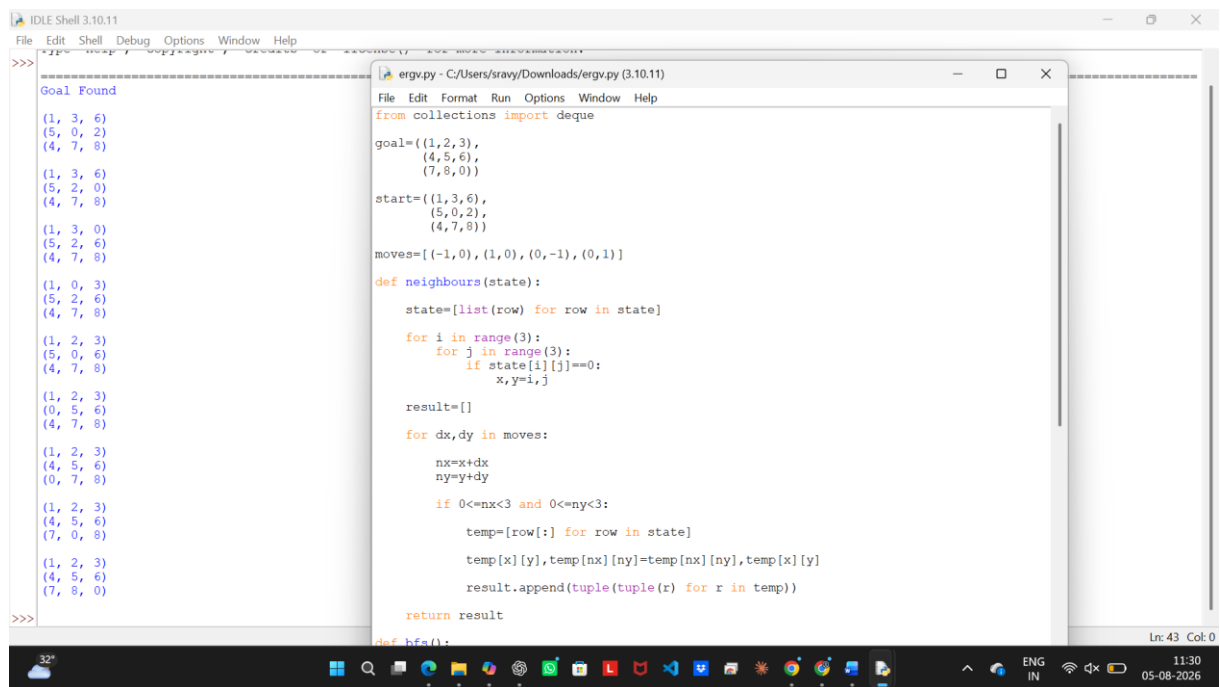
(Initial State)



Intermediate States



Goal State



```
Goal Found
(1, 3, 6)
(5, 0, 2)
(4, 7, 8)

(1, 3, 6)
(5, 2, 0)
(4, 7, 8)

(1, 3, 0)
(5, 2, 6)
(4, 7, 8)

(1, 0, 3)
(5, 2, 6)
(4, 7, 8)

(1, 2, 3)
(5, 0, 6)
(4, 7, 8)

(1, 2, 3)
(0, 5, 6)
(4, 7, 8)

(1, 2, 3)
(4, 5, 6)
(0, 7, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

>>>
```

```
ergv.py - C:/Users/sravy/Downloads/ergv.py (3.10.11)
File Edit Format Run Options Window Help
from collections import deque

goal=((1,2,3),
      (4,5,6),
      (7,8,0))

start=((1,3,6),
       (5,0,2),
       (4,7,8))

moves=[(-1,0),(1,0),(0,-1),(0,1)]

def neighbours(state):
    state=[list(row) for row in state]
    for i in range(3):
        for j in range(3):
            if state[i][j]==0:
                x,y=i,j

    result=[]

    for dx,dy in moves:
        nx=x+dx
        ny=y+dy

        if 0<=nx<3 and 0<=ny<3:
            temp=[row[:] for row in state]
            temp[x][y],temp[nx][ny]=temp[nx][ny],temp[x][y]
            result.append(tuple(tuple(r) for r in temp))

    return result

def bfs():
```

Result

The Breadth-First Search (BFS) algorithm was successfully implemented to solve the 8-puzzle problem. The algorithm explored the puzzle states level by level and found the shortest sequence of moves from the initial state to the goal state. This confirms that BFS guarantees the shortest solution for an unweighted 8-puzzle.